

**MODUL PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBYEK**

MODUL 4

Relasi Antarobject: Association, Aggregation, dan Composition



Disusun oleh:

Nirwana Haidar Hari, S.Pd., M.Kom.

**POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
PROGRAM STUDI DI LUAR KAMPUS UTAMA SUMENEP
PENS PSDKU SUMENEP**

2026

MODUL PRAKTIKUM P4

Relasi Antarobject: Association, Aggregation, dan Composition

1. Tujuan Praktikum

Setelah menyelesaikan praktikum, mahasiswa mampu:

1. Menjelaskan hubungan antarobject dalam sistem berorientasi objek.
2. Membedakan association, aggregation, dan composition.
3. Menentukan relasi antarclass berdasarkan studi kasus.
4. Mengimplementasikan object sebagai attribute pada class lain.
5. Menghubungkan beberapa class menjadi sistem sederhana.
6. Memperbarui class diagram proyek.
7. Mengembangkan *increment* proyek melalui Agile Sprint.

2. Review P3

Pada P3 mahasiswa telah memperbaiki class menggunakan:

- private,
- getter,
- setter,
- constructor,
- validasi data.

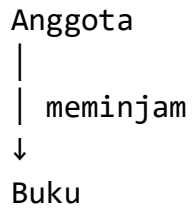
Contoh:

```
class Buku {  
  
    private String kode;  
    private String judul;  
  
    public Buku(  
        String kode,  
        String judul  
    ) {  
        this.kode = kode;  
        this.judul = judul;  
    }  
  
    public String getJudul() {  
        return judul;  
    }  
}
```

Pada P4, class tidak lagi berdiri sendiri.

Class mulai saling terhubung.

Contoh:



3. Konsep Singkat

A. Association

Association merupakan hubungan antarobject yang saling berinteraksi.

Contoh:

Mahasiswa ----- Dosen

Contoh implementasi:

```
class Mahasiswa {

    private String nama;
    private Dosen dosenWali;
}
```

Object Mahasiswa memiliki referensi terhadap object Dosen.

B. Aggregation

Aggregation menunjukkan hubungan **has-a**, tetapi object yang menjadi bagian masih dapat hidup secara mandiri.

Contoh:

Jurusan ◇----- Dosen

Dosen tetap dapat ada meskipun object Jurusan tidak ada.

Implementasi:

```
class Jurusan {

    private String nama;
    private Dosen ketuaJurusan;

    public Jurusan(
        String nama,
        Dosen ketuaJurusan
    ) {
        this.nama = nama;
        this.ketuaJurusan =
            ketuaJurusan;
    }
}
```

}

C. Composition

Composition merupakan hubungan yang lebih kuat.

Object bagian bergantung pada object pemilik.

Contoh:

Rumah ◆----- Ruangan

Ruangan dianggap sebagai bagian dari Rumah.

Contoh:

```
class Rumah {  
  
    private Ruangan ruangTamu;  
  
    public Rumah() {  
        ruangTamu =  
            new Ruangan(  
                "Ruang Tamu"  
            );  
    }  
}
```

Object Ruangan dibuat di dalam Rumah.

4. Perbandingan Relasi

Relasi	Hubungan	Ketergantungan
Association	uses-a	Lemah
Aggregation	has-a	Sedang
Composition	part-of	Kuat

Pertanyaan utama saat menentukan relasi:

Apakah object bagian masih dapat berdiri sendiri jika object utama dihapus?

Jika **ya**, kemungkinan aggregation.

Jika **tidak**, kemungkinan composition.

5. Contoh Studi Kasus

Sistem Perpustakaan

Class:

Anggota

Buku

Peminjaman

Perpustakaan

Kemungkinan relasi:

Anggota ----- Peminjaman
 Buku ----- Peminjaman
 Perpustakaan ◇--- Buku

Contoh implementasi:

```

class Peminjaman {

    private Anggota anggota;
    private Buku buku;

    public Peminjaman(
        Anggota anggota,
        Buku buku
    ) {

        this.anggota = anggota;
        this.buku = buku;
    }

    public void tampilkanData() {

        System.out.println(
            anggota.getNama()
        );

        System.out.println(
            buku.getJudul()
        );
    }
}
  
```

6. Agile Sprint – P4

Sprint Goal

Menghubungkan class utama proyek sehingga object dapat berinteraksi sebagai satu sistem.

Sprint Backlog

Gunakan project hasil P3.

Contoh:

ID	Sprint Backlog	Status
----	----------------	--------

SB-01	Mengidentifikasi relasi antarclass	To Do
SB-02	Memperbarui class diagram	To Do
SB-03	Implementasi association	To Do
SB-04	Implementasi aggregation	To Do
SB-05	Implementasi composition	To Do
SB-06	Membuat pengujian antarobject	To Do

Status:

TO DO
↓
IN PROGRESS
↓
DONE

7. Tugas Praktikum

Tugas P4 – Implementasi Relasi Antarobject

Gunakan **project hasil P3**.

Mahasiswa tidak membuat project baru.

8. Bagian A – Identifikasi Relasi

Gunakan minimal **4 class** dari project.

Buat tabel:

Class A	Class B	Relasi	Alasan
Anggota	Peminjaman	Association	Anggota melakukan peminjaman
Perpustakaan	Buku	Aggregation	Buku dapat tetap ada secara mandiri
Pesanan	DetailPesanan	Composition	Detail merupakan bagian dari pesanan

Minimal terdapat:

- 1 association,
- 1 aggregation,
- 1 composition.

Jika studi kasus tidak memungkinkan ketiganya, mahasiswa harus memberikan argumentasi.

9. Bagian B – Update Class Diagram

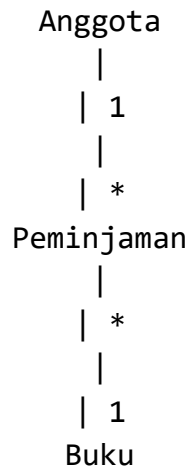
Perbarui class diagram project.

Diagram minimal menampilkan:

- class,
- attribute,
- method,
- multiplicity sederhana,
- association,
- aggregation,

- composition.

Contoh:



10. Bagian C – Implementasi Association

Implementasikan minimal satu association.

Contoh:

```

class Peminjaman {

    private Anggota anggota;
    private Buku buku;

    public Peminjaman(
        Anggota anggota,
        Buku buku
    ) {

        this.anggota =
            anggota;

        this.buku =
            buku;
    }
}
  
```

11. Bagian D – Implementasi Aggregation

Contoh:

```

class Perpustakaan {

    private String nama;
  
```

```

        private Buku buku;

        public Perpustakaan(
            String nama,
            Buku buku
        ) {

            this.nama = nama;
            this.buku = buku;
        }
    }

    Object dibuat di luar:
    Buku buku =
        new Buku(
            "B001",
            "Pemrograman Java"
        );

    Perpustakaan perpus =
        new Perpustakaan(
            "Perpus PENS",
            buku
        );

```

12. Bagian E – Implementasi Composition

Contoh:

```

class Pesanan {

    private DetailPesanan detail;

    public Pesanan() {

        detail =
            new DetailPesanan();
    }
}

```

Object DetailPesanan dibuat oleh Pesanan.

13. Bagian F – Pengujian Interaksi Object

Buat skenario minimal:

Test 1

Object berhasil dibuat.

Test 2

Dua atau lebih object dapat berinteraksi.

Test 3

Data dari object lain dapat digunakan melalui method.

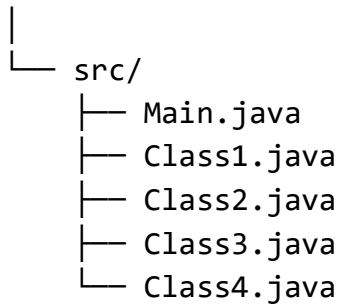
Contoh:

```
public class Main {  
  
    public static void main(  
        String[] args  
    ) {  
  
        Anggota anggota =  
            new Anggota(  
                "A001",  
                "Budi"  
            );  
  
        Buku buku =  
            new Buku(  
                "B001",  
                "Java OOP"  
            );  
  
        Peminjaman pinjam =  
            new Peminjaman(  
                anggota,  
                buku  
            );  
  
        pinjam.tampilkanData();  
    }  
}
```

14. Luaran Tugas

Kumpulkan:

```
P4_NRP_Nama/  
|  
└─ README.pdf
```



README.pdf maksimal **3 halaman**.

Halaman 1

- Nama project
- Sprint Goal
- Sprint Backlog
- Tabel identifikasi relasi

Halaman 2

- Class Diagram terbaru
- Penjelasan association
- Penjelasan aggregation
- Penjelasan composition

Halaman 3

- Screenshot hasil running
- Skenario pengujian
- Sprint Review
- Sprint Retrospective

15. Sprint Review

Item	Hasil
Class diagram diperbarui	...
Association berhasil	...
Aggregation berhasil	...
Composition berhasil	...
Object dapat berinteraksi	...
Program berjalan	...
Kendala	...

16. Sprint Retrospective

Jawab singkat.

What Went Well?

Bagian relasi apa yang berhasil diimplementasikan?

What Went Wrong?

Apa kesulitan dalam menentukan jenis relasi?

Improvement

Apa yang perlu diperbaiki pada desain class?

Maksimal **3–5 kalimat**.

17. Definition of Done

Tugas P4 selesai apabila:

- Menggunakan project P3.
- Minimal 4 class digunakan.
- Relasi antarclass telah dianalisis.
- Class diagram diperbarui.
- Minimal 1 association diterapkan.
- Minimal 1 aggregation diterapkan.
- Minimal 1 composition diterapkan atau disertai argumentasi jika tidak relevan.
- Object berinteraksi melalui method.
- Tidak mengakses attribute private secara langsung.
- Program berhasil dikompilasi.
- Program berhasil dijalankan.
- Minimal 3 skenario pengujian tersedia.
- Sprint Backlog diperbarui.
- Sprint Review tersedia.
- Sprint Retrospective tersedia.

18. Penilaian

Komponen	Bobot
Analisis relasi antarclass	15%
Class Diagram	20%
Implementasi Association	15%
Implementasi Aggregation & Composition	25%
Pengujian interaksi object	15%
Agile Sprint & dokumentasi	10%
Total	100%

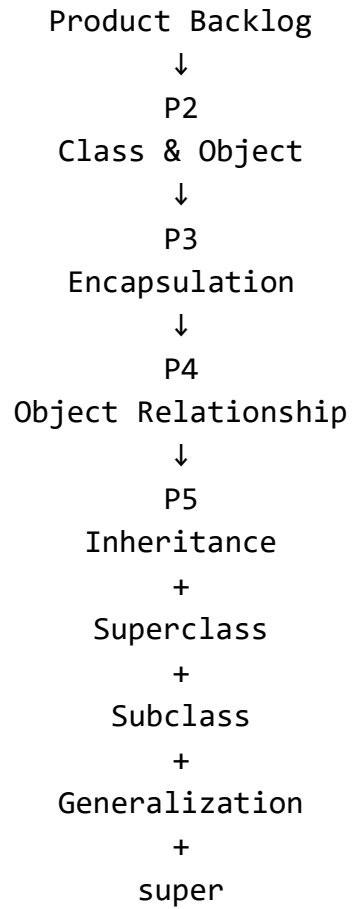
19. Target Sprint Berikutnya

Hasil P4 menjadi fondasi untuk **P5 – Inheritance dan Generalization**.

P1

Project Goal

+



Pada P5, mahasiswa akan mengevaluasi class yang memiliki attribute atau behavior serupa, kemudian melakukan **generalization** untuk membentuk superclass dan subclass.